Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Supporting Self-Descriptiveness, Reusability, and Code Bidirectionality in Spreadsheet Programming

Bachelor Thesis

Robin Frey

January 06, 2026

Advisors: Prof. Dr. Z. Su, Dr. S. Thorgeirsson
Department of Computer Science, ETH Zürich

Acknowledgments

You can use this section to thank whoever you want to in the context of your thesis. This can of course include family and friends, colleagues, your advisor, and in general anyone that provide some form of support. You can also take the opportunity here to give credit to people who may have enlightened you with discussions, and also certainly your co-authors of an academic paper related to the thesis, should there be one.

Abstract

Spreadsheet programs are one of the most widely used computational tools. Previous work identified that existing spreadsheet tools offer limited support for understanding formula logic and tracing dependencies. In particular, users spend a lot of time on information detours. This thesis presents a flow graph based visualization, which visualizes the formula dependencies as an interactive graph alongside the spreadsheet. By having the spreadsheet and the flow graph visualization side by side it aims to preserve the spatial context of the user in the spreadsheet while allowing the user to trace the dependencies of the formula even across different worksheets. To evaluate the usefulness, an explorative study was conducted with 5 participants, who had to solve three tasks and fill out the NASA-TLX, UEQ+ and answer three open-ended questions. The overall NASA-TLX mean was 27.7 out of 100 and the UEQ+ score indicates a positive user experience. In the open-ended questions the participants confirmed that they found the tool helpful to understand formulas and intuitive to use for navigating. However, the study was explorative and did not include sessions where particpants used only traditional spreadsheet tools. Therefore, to be able to correctly identify the benefits of the tool, future work should conduct a more extensive study.

Contents

Contents iii

1 Introduction 1
1.1 Motivation . . . 1
1.2 Problem statement . . . 1
1.3 Thesis outline . . . 2

2 Background 3
2.1 The spreadsheet paradigm . . . 3
2.1.1 Spreadsheet units . . . 3
2.1.2 Dependencies . . . 3
2.2 Cognitive dimensions . . . 4
2.3 Related work . . . 5
2.3.1 Built-in spreadsheet tools . . . 5
2.3.2 Separate dataflow visualizations . . . 5
2.3.3 Grid-embedded visualizations . . . 6

3 System Design 7
3.1 Architectural overview . . . 7
3.2 The presentation layer . . . 7
3.2.1 Layout strategy . . . 8
3.2.2 Flow graph justification . . . 8
3.2.3 Interaction design . . . 9
3.3 The visualization layer . . . 10
3.3.1 Literals . . . 10
3.3.2 Arithmetic operations . . . 11
3.3.3 Reference Nodes . . . 11
3.3.4 Functions . . . 14
3.3.5 Intermediate Results . . . 16
3.4 The parsing layer . . . 16

3.4.1 Parsing 16
3.4.2 Serialization 16
3.4.3 Transformators 16
3.5 The data layer 17
3.5.1 Formula engine selection 17
3.5.2 Single engine instance 17
3.5.3 Asymmetric data flow 17
3.5.4 Data Structure 18

4 Evaluation **19**
4.1 Study Design and Methodology 19
4.1.1 Participants 19
4.1.2 Tasks 19
4.1.3 Instruments 20
4.2 Quantitative Results 20
4.2.1 NASA-TLX Results 20
4.2.2 UEQ+ Results 21
4.3 Qualitative Results 23
4.3.1 Positive feedback 23
4.3.2 Frustrations 23
4.3.3 Improvement Suggestions 24
4.4 Study Discussion 24
4.5 Limitations 25

5 Conclusion **26**

Bibliography **28**

Dummy Appendix **30**

Chapter 1

Introduction

1.1 Motivation

Spreadsheets are one of the most common computer tools, with an estimated 1 billion user base [4]. This popularity is largely attributed to their accessibility. Users can store and organize their data in near instant time without writing traditional code. This allows for quick "what-if" projections of their data for decision-making [13]. Even though spreadsheets do not require writing code, users still have to write formulas and possibly use multiple spreadsheet at the same time. This hidden complexity can lead to weak points in the spreadsheets. Such possible weak points are called spreadsheet smells. Hermans et al., in *Detecting and visualizing inter-worksheet smells in spreadsheets* [7], identify common examples such as Feature Envy, where a formula accumulates excessive dependencies, and Inappropriate Intimacy, when data is tightly coupled across different Spreadsheets [7]. These smells hinder comprehension and make it harder for the user to find the relevant information. Ragavan et al., in *Spreadsheet Comprehension: Guesswork, Giving Up and Going Back to the Author* [12], find that readers of spreadsheet spend approximately 40% of their time "seeking information". This involves locating items, understanding interactions, tracing formula logic or understanding familiar spreadsheet concepts. This shows that it exists a need for tools to help spreadsheet users navigating and understanding the underlying complexity of Spreadsheets.

1.2 Problem statement

The study conducted by Srinivasa Ragavan suggests, that information detouring contributes to approximately 40% of the needed time to understand a spreadsheet. In some cases participants even failed to obtain the relevant information. Information seeking involves identifying dependents of

a cell, understanding the underlying logic of a certain value. Another time-consuming task is formula tracing. This occurs when a formula refers to another cell address, which again refers another and by doing so creating a chain of dependencies. Although modern spreadsheet programs highlight referenced cells on the grid, it is not sufficient when the cells are not visible in the grid [12]. These challenges highlight a gap in the current spreadsheet programs. While there is a clear need to improve the visualization for the logic of a spreadsheet, visualization alone is not enough. To address the problem of information detouring a tool must actively support the user in the navigation process, to make it easier to find relevant information while minimizing the context loss of the process.

1.3 Thesis outline

The remainder of this thesis is structured as follows. Chapter 2 provides the background necessary to understand the design decisions made throughout the thesis. It introduces the spreadsheet paradigm and its key concepts, the Cognitive Dimensions framework used to evaluate and motivate design choices, and an overview of related work in spreadsheet visualization. Chapter 3 presents the design of the proposed tool. The system is divided into four layers: the presentation layer, the visualization layer, the parsing layer and the data layer. Chapter 4 (Placeholder).

Chapter 2

Background

2.1 The spreadsheet paradigm

2.1.1 Spreadsheet units

A spreadsheet program is used to organize, store and maintain data and perform calculations on the data. The primary unit of a spreadsheet is the cell. This is the place where data is stored. A cell can have three types of content: a static value, a formula or an empty state. A value is data which is directly entered by the user. A formula is a recipe for calculating a value[2, p. 18]. For the thesis we define the terms worksheet and workbook as follows:

Definition 2.1 *A **worksheet** is a two-dimensional grid of cells that are organized into rows and columns. [2, p. 65]*

Definition 2.2 *A **workbook** is a collection of worksheets. [2, p. 65]*

Some functions use cell ranges as arguments. A cell range specifies a region of cells.

2.1.2 Dependencies

If a formula references other cells, it creates dependencies. The cell containing the formula is called a dependent of those referenced cells:

Definition 2.3 ***Dependents** of a cell are all cells whose formulas directly reference that cell.*

Conversely, the cells that the formula references are called the precedents of the formula cell:

Definition 2.4 ***Precedents** of a cell are all cells that the formula in that cell directly references.*

The dependencies can be tracked and modeled as a Directed Graph. With the constraint that spreadsheets do not allow cyclic referencing, the resulting graph will be a Directed Acyclic Graph. But spreadsheets programs do not explicitly show the dependency graph and the relationship between precedents and stay hidden. Such hidden dependencies can cause hidden changes. For example, if we have a formula in one worksheet, which is dependent on a cell in another worksheet, and change the data in this cell the change in the computed value of the formula stays hidden [5]. The change stays hidden because almost all spreadsheets only display one worksheet at a time.

2.2 Cognitive dimensions

To describe and evaluate certain design choices, this thesis uses vocabulary from the "Cognitive Dimensions" Framework from the paper *Usability Analysis of Visual Programming Environments: A 'Cognitive Dimensions' Framework* [5]. The following dimensions are particularly relevant:

Definition 2.5 ***Diffuseness*** *is defined as "How many symbols or graphic entities are required to express a meaning?" [5, p. 138]*

Diffuseness in this thesis is used to describe whether the flow graph requires many nodes to represent the logic and therefore is not compact.

Definition 2.6 ***Role expressiveness*** *is defined as "Can the reader see how each component of a program relates to the whole?" [5, p. 139]*

Role expressiveness is used to describe in the flow graph if the purpose of a certain component can be inferred from its appearance and how it interacts with the other nodes. If the flow graph has high role expressiveness the user should also be able to infer from reading the flow graph how the components relate to the underlying logic of the spreadsheet.

Definition 2.7 ***Hidden dependencies*** *are defined as "Is every dependency overtly indicated in both directions? Is the indication perceptual or only symbolic?" [5, p. 139]*

Green and Petre mention an example of hidden dependencies in spreadsheets: "The formula in a cell tells which other cells it takes its value from, but does not tell which other cells take their value from it." [5, p. 154]. This thesis evaluates whether the flow graph is able to show these dependencies and make them visible.

Definition 2.8 ***Viscosity*** *is defined as "How much effort is required to perform a single change?" [5, p. 139]*

Viscosity is relevant to evaluating and describing if the edit mode is easy and intuitive to use.

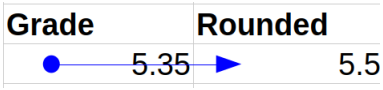

Figure 2.1: Tracking a precedent of a cell in LibreOffice Calc

2.3 Related work

2.3.1 Built-in spreadsheet tools

Excel and LibreOffice Calc have their own built-in tool to track dependents and precedents of a cell. If a user tracks for example the precedent of a cell (figure 2.1), the program draws an arrow of the precedent to the cell. Even though the arrows highlight the precedent / dependents of a cell, if they are located on a different worksheet the user has to move to the other worksheet and possibly lose their current spatial context. Another issue is that the user only sees the precedents / dependents which are in the viewport. If they are not located in the current viewport the user has to scroll away to locate the precedents / dependents.

2.3.2 Separate dataflow visualizations

Leveled dataflow

Several researches have proposed different approaches to display the underlying dependency graph of spreadsheets. Hermans et al. present in their paper *Supporting Professional Spreadsheet Users by Generating Leveled dataflow Diagrams* [6], a solution to explicitly show these relationships. Their approach consist by using three Views: The global view, the worksheet view and the formula view. The global view (figure 2.2) shows the relationships between the worksheets. The worksheet view groups the data in a worksheet into blocks and how they relate to each other. On the lowest level is the formula view which gives more detail about the calculations of a specific formula. One of the limitations of the leveled dataflow is, that it is a read only tool. It does not exist bidirectionality to allow the user to connect the data blocks to the data in the worksheet. Unlike the multiple representation spreadsheet discussed below, it does not allow edits to the view.

Multiple representation spreadsheet

To tackle the inability to clearly see the underlying computational structure and lack of abstraction, Sarkar et al. present their solution, multiple representation spreadsheet, in the paper *Calculation View: multiple-representation editing in spreadsheets* [10]. Their solution leaves the grid and formula syntax untouched, but it should provide additional representation to abstract. Their additional representation is a textual view, which describes "a set of formula

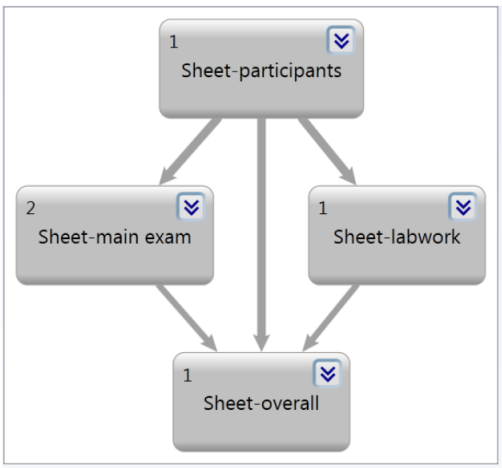

Figure 2.2: An example global view [6]

assignments" [10, p. 86]. The tool also allows for bidirectionality. The user can make an edit in one view, and it gets propagated to the other, it is live.

2.3.3 Grid-embedded visualizations

Fluid Visualization

In the paper *Fluid Visualization of Spreadsheet Structures* [9] by Igarashi et al. embed the visualization directly into the spreadsheet grid. They use different techniques for the visualization. The first one is the transient local view, which allows the user to see the incoming and outgoing cells of a selected cell. It is called transient because the view appears and gradually fades dependent where the cursor is moved to. It also implements a static global view, which shows the entire dataflow graph at once to the user. To highlight the direction of the dataflow, the system animates the dataflow.

Chapter 3

System Design

3.1 Architectural overview

The system can be divided into four layers.

1. The presentation layer
2. The visualization layer
3. The parsing layer
4. The data layer

The **presentation layer** is responsible for rendering the split view and dispatching user interactions. The second layer, the **visualization layer**, bridges the underlying formula strucutre and the flow graph by transforming the AST produced by the parsing layer into an interactive graph representation. The **parsing layer** is responsible for two conversions: parsing formula strings into an AST, and serializing an AST back into a formula string. The foundation of the system is the **data layer** which hosts the spreadsheet calculation engine and serves as the single source of truth for all spreadsheet data.

3.2 The presentation layer

The presentation layer is responsible for rendering the split-view and dispatching user interactions. User interactions include for example highlighting cells on hover, modifying constant values, and editing edges in the flow graph. The system uses a split view, where the worksheet occupies the left side and the flow graph occupies the right side, allowing both to be visible simultaneously.

3.2.1 Layout strategy

The spatial arrangement of the tool and the spreadsheet has a significant impact on the cognitive load during the spreadsheet comprehension.

Definition 3.1 *Juxtaposability is the ability to see any two portions on screen side-by-side at the same time [5].*

The split view

To achieve juxtaposability (Definition 3.1) between the spreadsheet and the flow graph, the system employs a split view layout. The spreadsheet occupies the left side, while the flow graph appears on the right side. The split view layout introduces the cognitive cost known as the split-attention effect. Chandler and Sweller, in *The Split-Attention Effect as a Factor in the Design of Instruction* [3], describe this as a phenomenon in which learners are forced to mentally integrate information from disparate sources. However, the split view is a response to a fundamental limitation of overlay-based approaches. As mentioned in section 2.3.1, traditional spreadsheet tools for tracing precedents and dependents use an overlay strategy. An overlay strategy ties the dependency visualization directly to the current worksheet view. But it fails when the user navigates to another worksheet to inspect a referenced cell, the visualization disappears, causing the user to lose the visual context of the formula they were investigating. This is the loss of spatial context identified in section 1.2, where users lose track when navigating between dependents and precedents. Furthermore, an overlay would occlude cells and formulas the user needs to see. By decoupling the flow graph from worksheet navigation, the split view keeps the dependency graph of the synchronized formula visible and stable while the user is able to freely navigate between worksheets.

3.2.2 Flow graph justification

The decision to represent the dependency graph of a formula as a visual flow graph was motivated by the need to expose a structure that exists implicitly in every formula but remains hidden from the user. Rather than requiring the user to read and mentally parse individual formulas, a visual flow graph makes the dependencies directly visible. Hermans et al. demonstrate that dataflow diagrams help users understand spreadsheet structure across multiple levels of detail [6]. The system in this thesis focuses on the formula level exclusively, as supporting multiple levels of abstraction would have added significant complexity. The flow direction is left to right, mirroring the spatial convention of spreadsheets where formula inputs are located in columns on the left side of a formula cell.

3.2.3 Interaction design

Before interacting with the flow graph, the user must first synchronize it to a formula. When the user selects a cell containing a formula in the worksheet, the flow graph view exposes a synchronize button. Clicking it loads the selected formula's dependency graph into the flow graph view. The flow graph serves as a visualization of a formula's dependency graph and as an interface for exploring and manipulating it. The interface operates in two modes, the preview mode and the edit mode, where edit mode extends the available interactions to include modifications to the underlying data. In edit mode the flow graphs border is orange whereas in preview mode it is a blue border. The visual distinction between modes serves as a cue to the user that changes are now possible. Edit Mode is intentionally easy to enter, prioritizing a low viscosity (definition 2.8).

Preview Mode

Preview Mode is the default state of the flow graph and is designed for exploration. Hovering over a reference node highlights the corresponding cell or range in its worksheet. If the cell is already visible in the current viewport, this allows the user to see the dependencies at a glance without disrupting their current position in the worksheet. Clicking on a reference node explicitly scrolls to the corresponding worksheet and cell or range, enabling closer inspection when the user requests it. This aims to preserve the user's spatial context in the worksheet and only disrupt it on demand.

Edit Mode

Edit Mode extends the interactions available in Preview Mode to allow direct manipulation of the flow graph and its underlying data. It is designed to minimize the distance between recognizing a value to change and modifying it. Rather than requiring the user to trace the node back to its source in a worksheet and modify it there, changes can be initiated directly from the point of interest in the flow graph, aiming to reduce viscosity [5]. Edit Mode is always entered through an explicit user action:

1. clicking on the edit button
2. double-clicking on an inline value or address
3. double-clicking an edge
4. dragging from a connection handle

The entry action determines the initial editing context. Double-clicking on a node's address allows the user to select a new address directly from the worksheet. This interaction is designed to leverage closeness of mapping to already known spreadsheet tools [5]. In most spreadsheet tools the user

can select cells and ranges directly in the workbook to supply them as function arguments. Double-clicking an inline value transforms the field into an editable input, enabling the user to modify the constant in place. In both cases, the change is committed by pressing Enter or clicking the confirmation button. Double-clicking an edge enters Edit Mode with that edge pre-selected for further manipulation. Dragging from a connection handle initiates edge creation or modification. As the user drags from a source handle toward a target handle, invalid handles are grayed out to constrain the action to permissible connections. If the target handle has no existing connection, the new edge is created directly. If the target handle is already occupied, a popover prompts the user to either replace the existing edge or swap positions with it. The swap option aims to minimize the edit steps for especially switching arguments inside a function. Clicking the edit button enters the Edit Mode without an initial editing context. Once in edit mode, the user may freely select any element to begin editing.

3.3 The visualization layer

In the visualization layer the system transforms the AST into a graph representation which is then rendered as an interactive flow graph. This section focuses on the visual design choices of the flow graph nodes and how each AST node maps to a corresponding graphical representation. The primary goal when transforming the AST is to reduce the overall number of nodes and edges while improving clarity. This aims to create a compact and informative flow graph. Literal nodes are inlined into their parent nodes. Arithmetic operations are either transformed into a binary operation node or collapsed together into one expression node. Reference nodes are enhanced with visual cues and header inference heuristics. Function nodes are augmented with semantic argument labels. Conditional functions are represented by a separate node to help the comprehension of the branching logic. Function nodes as well as reference nodes display their intermediate results to provide more information about the current state to the user.

3.3.1 Literals

Literals are for example numbers or strings, literals are used almost in every formula. Because the meaning of literals are evident rendering them as nodes would not add meaningful help for the users. The system solves this by inlining literals into their parent node. There they are still visible but do not occupy any additional space in the graph.

3.3.2 Arithmetic operations

The system identifies certain operations as simple operations. These include arithmetic operators, string and comparison operations.

Binary operation node

If the left and right operand of these operations are not simple operations themselves, the system creates a binary operation node. Because these operations are self-explaining, the system aims to minimize the size of the node. Therefore, the system only displays the operator symbol with pseudo operators or inlined constants.

Expression nodes

When multiple operations are chained together the system creates an expandable expression node. Initially, this node displays only a preview of the complete expression, occupying minimal space in the flow graph while still indicating to the user that here an operation is happening. Users can expand the node via double click to see the complete expression. Like the expandable cell reference node (section 3.3.3) this design follows a details-on-demand strategy to balance between compactness and accessibility. In its minimized state, the expression node serves as a visual indicator that a calculation occurs at this point in the flow graph, without occupying too much space in the flow graph. However, this introduces a trade-off: users who want to examine the complete expression must perform an additional step compared to a fully-expanded default state. By grouping chained operations together, the system addresses two cognitive dimensions simultaneously. First, it reduces diffuseness (definition 2.5) by only having one node instead of multiple nodes. Second, it improves role expressiveness by presenting these operations in a more familiar way. Because these simple operations are more self explaining than functions, it is potentially easier for the user to read the familiar inline expression compared to a series of binary operation nodes.

3.3.3 Reference Nodes

In order to understand the dependencies of the formula the system explicitly creates nodes for references. A reference node has three important components: Address, label and result. The most important component is the address component. The address is colored accordingly to the pre-defined color of its worksheet. When importing a new workbook the system generates distinct colors for each worksheet. The coloring gives a fast visual cue to the users where nodes reside. For example when formulas refer across worksheets two distinct addresses can look the same even though they refer two different cells or ranges (figure 3.1).

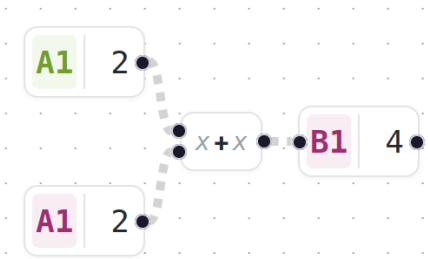

Figure 3.1: Two references resolve to the same looking address even though they do not reside in the same worksheet. The colors indicate that A1 (green, top) is on a different worksheet than A1, B1 (pink).

Figure 3.2: In the sheets tab of the data table view the corresponding worksheets are colored corresponding to their assigned color.

Label

Hodnigg et al. identify in *Comprehending Spreadsheets: Which Strategies do Users Apply?* [8], the comprehension indicator "realization" which refers to an insight of the spreadsheet to the user. And one of the comprehension patterns is reading a label[1] followed by a "realization". This indicates that labels can help in the process of comprehending spreadsheets. To help the user in the process of reading labels, the system implements a header inference heuristic. One limitation of this heuristic is that it assumes well-structured spreadsheets and may yield suboptimal results in spreadsheets that do not follow standard column/row orientation. To generate a label the system scans upwards and leftwards and selects the first non-formula string value to compose them into a composite label. For example in figure 3.3 the system resolved J4 to Bob:Rounded. By inferring the label for the user, the system aims to preserve the user's spatial context within the data table, as the users are no longer required to visually scan through columns and rows.

Expandable Cell References

As mentioned in section 1.2, one major challenge for spreadsheet comprehension is the tracing of the dependency chain. To address this the system implements a recursive resolution strategy, to build the dependency chain. To not overwhelm a user when presented the whole dependency chain of the synchronized cell, the system adopts a details-on-demand [11] strategy. Upon synchronizing a new cell, the expandable cell reference nodes start in a minimized state (figure 3.3). Expandable cell reference nodes (figure 3.4) are only expanded upon explicit user interaction. With this strategy the system aims to reduce the Diffuseness of the flow graph. Crucially the graph navigation is decoupled from the grid navigation. When a user expands a cell

[1] textual descriptions in cells

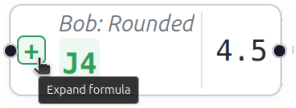

Figure 3.3: Expandable cell reference node when minimized

Figure 3.4: Expandable cell reference node when expanded

reference node the viewport of the worksheet remains the same. This aims to preserve the spatial context of the user in the worksheet while being able to trace down the dependency chain, directly addressing an issue reported by Srinivasa et al., where a user has lost track when navigating between dependents and precedents [12].

Merging references

In Spreadsheets it is common that formulas use the same references. The flow graph tries to merge same reference nodes, which are located in a distance of 5 edges, into one single node. The distance of 5 edges was chosen such that only logical near nodes can be merged and that maybe identical nodes but which are located at a complete different location in logical terms do not disrupt the flow graph. Merging nodes together, makes the flow graph more compact.

Range references

In difference to cell references, which hold one value, range references encompass many cells with many value. The system cannot display all the cells and their values of the range initially without clustering the flow graph. To reduce the diffuseness the system implements, similar to the expression nodes and the expandable reference nodes, the details on demand strategy. If the user wants to know or edit a value of the range, they first must expand the range node. When expanded the user can edit values like static values or the values in cell reference nodes.

3.3.4 Functions

A function in Spreadsheet programs is a predefined formula, which takes one or more arguments to produce a result [2]. In traditional formula syntax, the semantic meaning of an argument is only defined by its position for example in the function: VLOOKUP(E6,H6:I9,2,0). When reading this the user must actively recall which argument is responsible for what. This mental mapping increases cognitive load. To solve this the system gives each argument a semantic label. By visually linking a node to a semantic label the system aims to increase the role expressiveness (definition 2.6) of the argument nodes. This should decrease the need of consulting a documentation.

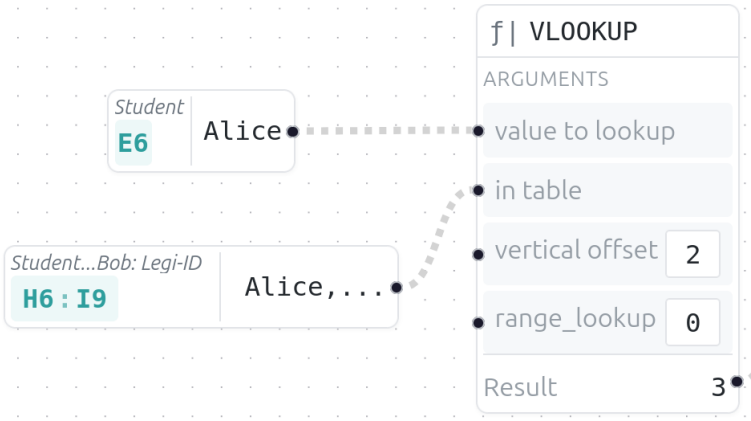

Figure 3.5: Example of the linking between argument nodes to semantic argument labels

Conditional Nodes

Conditional functions[2] introduce branching logic to Spreadsheets and add additional challenges for formula tracing. First, as mentioned in the section above, conditional arguments also suffer from the positional semantics problem. Furthermore, conditional functions in textual form suffer from the lack of runtime visibility. Users must mentally simulate the boolean condition to determine which branch is active. To address this, the system only displays the currently active branch and minimizes the inactive branches at first. This also helps to reduce the diffuseness of the flow graph but hides some logic as tradeoff. To highlight the active branch to the user, the system implements the following strategies:

1. Minimization of inactive branches
2. Visual dimming

[2] e.g IF, IFS

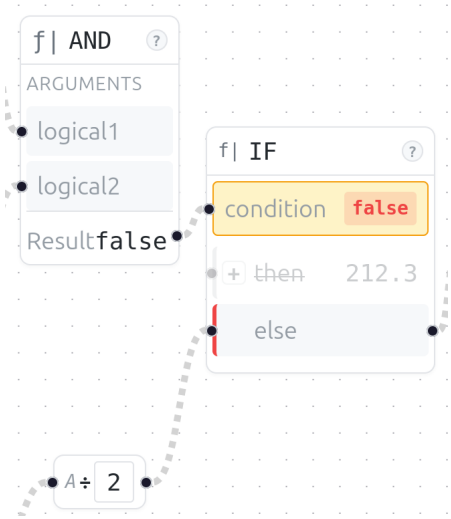

Figure 3.6: The system strikes non taken arguments in conditional nodes through

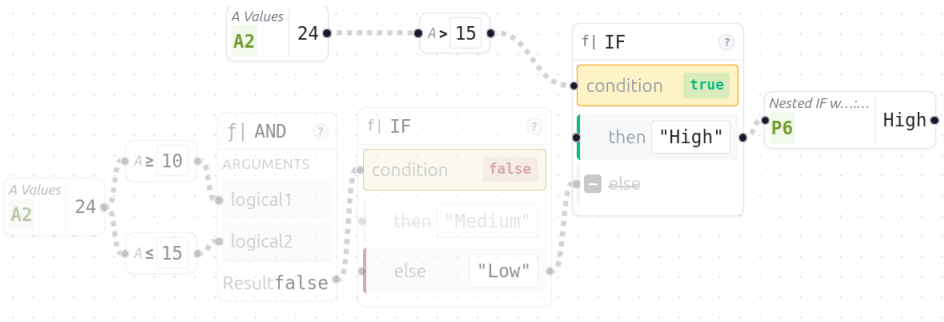

Figure 3.7: The system grays non taken branches out if expanded

3. Highlighting the contributing argument

As mentioned above the system minimizes inactive branches by default. If the users still wants to explore the logic of the inactive branch they can expand the subflow by clicking on the plus symbol. This mechanic is identical to the one of expandable cell references as mentioned in section 3.3.3. In order to still have a visual difference between the active and inactive branches, when the user expands inactive branches, the system dims the inactive branches (figure 3.7). And to reinforce this state in the node itself the inactive arguments are struck through (figure 3.6). These visual cues aim to prevent misinterpreting an inactive value as contributing factor to the result of the conditional node. Furthermore, because spreadsheets are dynamic the system updates the active branch when dependents change.

Helper Button

Ragavan et al., in Spreadsheet Comprehension: Guesswork, Giving up and Going Back to the Author [12], pinpoint one of the information needs is the "understanding of unfamiliar concepts". This need happens when users encounter unfamiliar functions and do not understand what they calculate. To help the user as an intermediate step before the user has to go to the documentation, every function node has a help butotn. The help button summarizes based on the input value and result value what is calculated in this function node.

3.3.5 Intermediate Results

Another comprehension pattern Hodnigg et al. identify is that after reading values realizations can follow [8]. This is one of the reasons why the system displays the intermediate results of reference nodes or functions. Intermediate results in reference nodes refer to the content of the cell or range while in function nodes to the calculated output of the function. The second reason the system explicitly shows the intermediate results is to allow the users to verify each sub-step in the Flow Graph.

3.4 The parsing layer

The Parsing Layer's main responsibility is transforming a spreadsheet formula into an AST, and its inverse operation serializing the AST back into a formula. This allows the system to transform and reconstruct formulas after graph-based edits.

3.4.1 Parsing

The system uses Chevrotain, a parser building toolkit for JavaScript. The grammar follows standard spreadsheet formula syntax, supporting basic operations, cell references, ranges and function calls.

3.4.2 Serialization

The serializer reconstructs a formula string from an AST. This is needed when a user has made an edit in the flow graph, because the modified AST has to be written back to the spreadsheet cell.

3.4.3 Transformators

Whenever a user applies an edit in the flow graph the system locates the corresponding AST nodes and applies the corresponding transformation. All transformations are immutable, which means that the system produces a

new AST rather than modifying the original. The transformed AST is then serialized and written back to the cell.

3.5 The data layer

The data layer forms the foundation of the system architecture. It serves as the single source of truth for all spreadsheet related data. This includes for example: cell values, formulas and computed results.

3.5.1 Formula engine selection

For the formula engine it is important that it provides a lot of functions which are also used in Microsoft Excel or Google Sheets to make the user as familiar with the formulas as possible. Another important feature should be the ability to handle multiple worksheets. The system uses HyperFormula as its calculation engine. HyperFormula provides compatible syntax with Microsoft Excel, 400 built-in functions and an easy integration with custom front-ends [1]. This allows the user to transfer their pre-knowledge about spreadsheet functions and behavior to this system.

3.5.2 Single engine instance

A single HyperFormula instance maintains the whole calculation state for the entire workbook. This allows the user to reference cells and ranges from different worksheets. Having only one single-instance implies having only one dependency graph for all worksheets. This simplifies the process of checking and updating dependencies. But this also implies that one computationally heavy formula could affect the responsiveness of the entire workbook.

3.5.3 Asymmetric data flow

As mentioned in the presentation layer 3.2 each worksheet is implemented by their own Handsontable component. Each component reads/writes from/to the same Hyperformula Engine. All components of the Graph-View also read directly from the same Hyperformula Engine. A read is done for example when nodes calculate their intermediate results. In contrast to the read paths, data manipulation in the Graph-View follows a slightly different path. Because HyperFormula is a headless calculation engine it is not able to know if it is connected to any Handsontable component. Therefore to prevent inconsistencies between the datatable and the underlying calculation engine, edits to the data have to go through the active Handsontable component to trigger rendering and the corresponding hooks.

3.5.4 Data Structure

HyperFormula recognizes five basic cell value types: numbers, booleans, dates, strings and empty cells. Dates are stored internally as a numeric representation rather than as date objects [1].

Chapter 4

Evaluation

4.1 Study Design and Methodology

4.1.1 Participants

Five participants completed the full study. Participants varied in their self-reported spreadsheet expertise, which was measured on a ten-point scale (0=no experience, 10=expert), with a mean expertise of 3.4.

4.1.2 Tasks

The Participants completed three spreadsheet comprehension tasks of increasing complexity.

Task 1 - Sales Commissions

This is a single worksheet in which each employee's final commission is computed through a normal calculation chain, involving VLOOKUP and IF functions. Five questions to this task were asked, about comprehension, prediction and error correction.

Task 2 - Student Grading

Again this is a single worksheet but with many IF statements and logic conditions. The six questions of this task aimed to test the understanding of conditional logic. In difference to the other tasks only one student's data was populated in order to focus even more on the conditional logic.

Task 3 - Inventory Pricing

Opposite to the other tasks, this workbook contained 3 worksheets with pricing analysis, product catalogue and supplier costs.

Task	Mental Demand (Average)	Temporal Demand (Average)	Performance (Average)	Effort (Average)	Frustration (Average)
Task 1	40.0	10.0	38.0	42.0	20.0
Task 2	24.0	10.0	37.0	35.0	10.0
Task 3	42.0	21.0	31.0	41.0	15.0
All Tasks combined	35.3	13.7	35.3	39.3	15.0

Table 4.1: Average NASA-TLX subscale scores per task

4.1.3 Instruments

Three different instruments were used to collect study data:

NASA-TLX

The NASA Task Load Index is a widely used multidimensional rating procedure for assessing perceived workload. In this study the physical demand was omitted due to the reason that it is not meaningful for a screen-based study. The five collected subscales were mental demand, temporal demand, self perceived performance, effort and frustration. Each of them had to be rated on a 0-100 scale. After each task the participants had to answer the same NASA-TLX without the physical demand, in total 3 times.

Custom UEQ+

The study used a custom variant of the user experience questionnaire plus and the participants had to answer it after they completed all three tasks. The questionnaire consisted of seven scales, Usefulness, Dependability, Efficiency, Perspicuity, Structure, Usability and Visual Aesthetics. Each of these scales had four items on a seven-point Likert scale (1=most negative pole; 7=most positive pole).

Open-ended questions

At the end the participants were invited to answer three open ended questions:

1. What they liked most about the tool
2. What they found frustrating about the tool
3. What they would improve

4.2 Quantitative Results

4.2.1 NASA-TLX Results

The NASA-TLX results across the three spreadsheet tasks indicate that the users experience a low to moderate cognitive load when using the tool. The

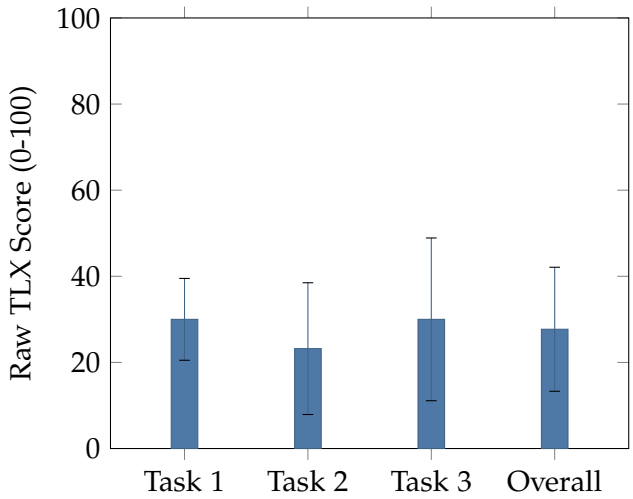

Figure 4.1: Raw TLX Scores for each Task and Average over all Tasks

overall Raw TLX, an unweighted variant of the NASA Task Load Index that averages the subscales without a pairwise importance weighting, was 27.7 (SD = 11.8) on a 0-100 scale. Table 4.1 presents the scores for all NASA-TLX dimensions across the three spreadsheet tasks, while figure 4.1 only presents the overall Raw TLX score of each task with their respective standard deviation. This result is encouraging, because it indicates that the tool did not put an excessive cognitive burden on the participants, even when asked relative complex spreadsheet tasks. Task 2 achieved the lowest mean score of 23.2 (SD = 15.3), while Task 1 and Task 3 both achieved the highest mean score of 30.0 (SD = 9.5 and 18.9 respectively). The frustration levels were comparatively low, with means of 20.0, 10.0 and 15.0. This could indicate that even when participants encountered difficulties, that the tool was able to help them make some progress in the comprehension and that the interactions with the flow graph were intuitive. Important to mention is, that between the individual participants there was substantial variability across all tasks. For example, participant 5 consistently reported very low cognitive load (overall means of 16.0, 5.0, 8.0) whereas participant 2 reported a significant higher load with overall means of 29.0, 47.0 and 51.0. One reason for this variability could be the difference in spreadsheet knowledge, but the small sample size does not allow for generalizability.

4.2.2 UEQ+ Results

The UEQ+ scores indicate an overall positive user experience with the tool across different quality dimensions. Figure 4.2 presents the mean scores and standard deviations for each UEQ+ scale. All seven subscales received a positive mean score, ranging from 1.45 up to 1.95. The subscores Efficiency

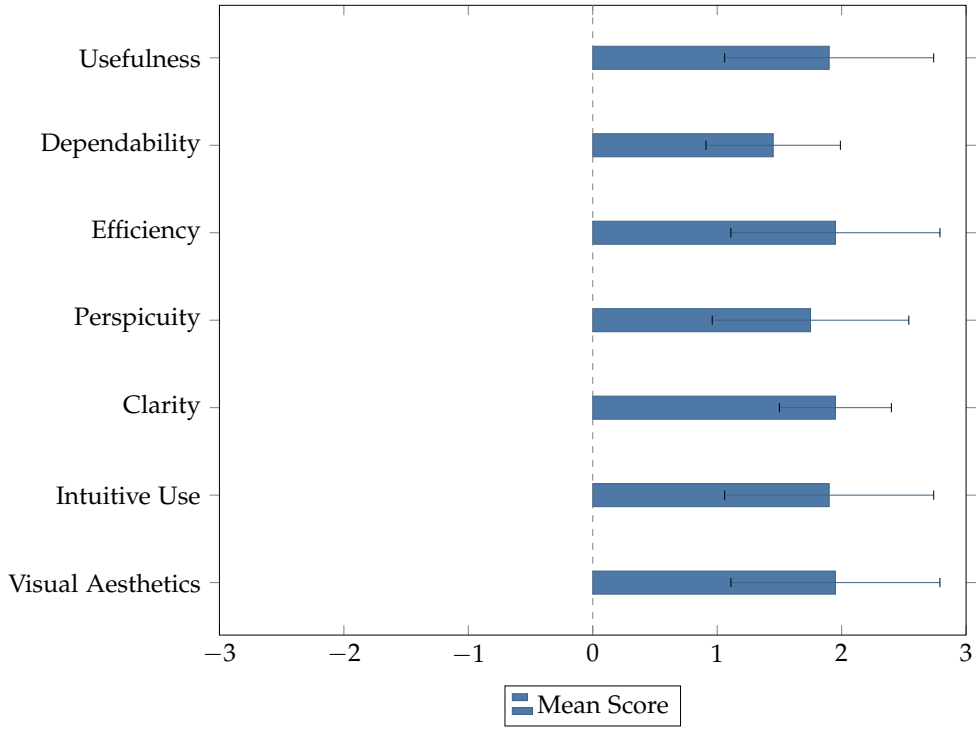

Figure 4.2: Mean scores of the UEQ+ sclaes

and Visual Clarity achieving the highest scores, both with mean scores of 1.95 (SD = 0.84 and SD = 0.45 respectively). The goal of the tool was to improve the self-descriptiveness of Spreadsheets and the results suggest that the tool was perceived both as practical and goal-supportive. Especially the high usefulness score (M = 1.90) combined with its high importance rating (M = 6.20 out of 7) indicates that the participants found the tool useful and also be highly relevant for this type of tool. This is outcome is encouraging, because it indicates that the tools is well received in the dimensions that matter the most to users. The perspicuity scale (M = 1.75), which describes how easy it is to learn and understand a tool, received the highest importance rating among all scales (M = 6.40), indicating that participants value if the tool is easy to learn and understandable. The score itself is positive, but has a notable variabiltiy (SD = 0.79), with one participant rating the perspicuity of the tool substantially lower (0.50) than others. This participant also self reported the lowest spreadsheet knowledge level of 1 and noted in the custom-questions that function arguments sometimes felt like a "blackbox". This could indicate that the tool has a steeper learning curve for beginners than intermediates. Dependability (M = 1.45) received the lowest mean score among all subscales, though it remained in the positive range. Dependability descirbes if the tool behaves predictably and meets user expectations. This lower score can be attributed to some observed instances during the study, in which participants

tried to edit cell reference nodes values even though they are formulas and therefore read-only. This result's indicates that there exists a gulf of execution, where the user's intended action does not map directly onto the system's interaction. A more predictable interaction would be if the system allows for formula containing reference nodes, like non formula contianing reference nodes, the ability to directly manipulate the value, even if it destroys the formula. The aesthetic scale (M = 1.75) received the lowest importance rating (M = 4.60), suggesting that the participants cared more about if the tool helps in achieving their goal than how it looks. Nevertheless, the positive result indicates that the participants liked the aesthetics of the tool. Also the aesthetic scale has the highest variability of all subscales (SD = 0.90) which could reflect the individual preferences for visual design.

4.3 Qualitative Results

4.3.1 Positive feedback

After the NASA-TLX and UEQ+ questionnaire the participants were invited to fill out the three open-ended questions. For the first question: "What did you like the most about the tool?", the most mentioned strength was that the tool helped the user to understand the logic behind cells. Participant 1 writes:"it helped me understand the hidden meaning of the numbers of the sheet". Which aligns with the tool's goal.

Another major theme was the navigation and flow graph. This corresponds a lot with the Intuitive Use scale of the UEQ+ questionnaire, which achieved a median of 5.80. Participant 5 mentions that:"The flow chart made it very easy to understand the logic behind each cell. Especially the navigation by double-clicking was very intuitive."

4.3.2 Frustrations

All the participants had different frustrations. One theme which was mentioned by two particpants was that opening all the expandable reference nodes could be irritating. Participant 4 writes: "in a long tree, sometimes difficult to understand interdependacies". But Participant 4 is not sure if this is caused by the tool or by the nature of the more compex formulas. Another concern of Participant 1 was the missing of a clear button. Participant 1 wished to return to a clean state after deep exploration. The Participant with lowest spreadsheet knowledge, Participant 3, mentions that the arguments of functions were a bit frustrating and that they felt a bit like blackboxes.

4.3.3 Improvement Suggestions

The participants offered different improvement suggestions. Based on the frustration of Participant 1, Participant 1 would add a clean button in order to get a clean state of the tool on demand. Participant 2 suggested making expansion points more visible and also highlight expansion points when "unclicked" and gray when clicked. This is the reverse behavior of the current implemented highlighting. Participant 2 also proposed to make the flow graph view and worksheet view adaptable. The users should be able to decide how much worksheet view or flow graph view they need. Participant 3 would improve the description of the question mark boxes and add more verbose description in order to explain in more detail what the function does. Participant 5 proposed that clicking on a VLOOKUP result should show the user where exactly the result was found.

4.4 Study Discussion

The low-to-moderate NASA-TLX scores suggest that the tool does not impose heavy cognitive demands, while the positive UEQ+ scores indicate a positive user experience across different dimensions. The custom questions help explain some results of the NASA-TLX and UEQ+ scores.

The central finding of this study is that the flow graph fulfills the design goal of making the spreadsheet formula logic more accessible and more navigatable. Another finding is that despite encountering specific usability problems, as mentioned by participants in the qualitative study, that the frustration ratings on the NASA-TLX were low. Several participants encountered confusion when attemtping to directly edit formula output values in the flow graph, which the tool did not support. Yet, this did not have a big impact on the overall satisfaction, as seen in the UEQ+ Usefulness score and the general positive answers in the custom questions.

Nevertheless, the study also reveals ares where the design of the tool could be refined. The relative lower score, 5.45, of Dependability compared with the other scores, indicates that there is a gap between the users expected behavior and what the tool allows them to do, gulf of execution. In the study it was observed, that some users for example expected to be able to modify every value within the flow graph, but the system required them to navigate back to the worksheet and modify it there. This gap could be easily fixed by allowing the user to be able to change these values even though it potentially could destroy the current formula. Similarly, the feedback in the custom questions about the expansion point visibility and the interaction intensive task of expanding all reference nodes suggest shortcuts or more efficient traversal of the flow graph in deeply nested formulas. The next

design iteration could for example introduce a collapse all or an expand all button.

Finally, all participants, regardless of prior spreadsheet knowledge, were able to complete the comprehension and manipulation tasks and indicates the tool's accessibility. Also the perspicuity importance rating of 6.40, which was the highest amongst all other UEQ+ scores, underscores that for the users it is very important that the system is easy to learn. For future iterations it should be a design priority to maintain or even improve the learnability of the system, especially for users who may not understand function arguments or lookup semantics.

4.5 Limitations

There exist several limitations when using the findings of this study. First is the small sample size of five participants. This small sample constrains the generalizability of the results. The standard deviations observed across the scores indicate high differences between individuals.

Secondly, the spreadsheet knowledge level was relatively homogeneous. Only participants with spreadsheet knowledge between 1 and 6 took part in the study and the study did not assess how well expert level users perform with the tool.

Another important limitation is that the study did not have sessions in which the particpants were only allowed to use traditional spreadsheet programs. This makes it impossible to know if the positive outcomes can be attributed to the flow graph visualization rather than other factors such as task familiarity, overall too easy tasks or just the Hawthorne effect. Finally, the qualitative observations recorded during the study session could potentially also introduce some degree of observational bias.

Chapter 5

Conclusion

This thesis presented a flow-graph-based tool to help users comprehend and manipulate spreadsheets. The tool aims to address a core difficulty identified in prior work: that spreadsheet users spend a big portion of their time seeking additional information, tracing formula dependencies and trying to understand the underlying logic of cell values [12]. The tool addresses this by giving the user a flow graph view alongside the spreadsheet view. The design of the tool was guided by the cognitive dimensions framework [5], especially regarding the hidden dependencies, viscosity and diffuseness dimensions. The flow graph visualization makes the hidden dependencies of a formula explicit and allows users to interact with the flow graph to help in the navigation and editing process without losing their spatial context.

An explorative user study with five participants evaluated the tool across three different spreadsheet tasks involving comprehension and manipulation. The overall mean NASA-TLX score was 27.7 of 100 (SD = 14.4), where the frustration subscale achieved one of the lowest results of the subscales with a mean of 15.0 of 100 (SD = 15.0). The results of the UEQ+ questionnaire suggest an overall positive feedback, with the Efficiency and Clarity subscales receiving the highest scores, both scoring a mean of 5.95. Additionally, in the open-ended questions of the study, the participants confirmed that the flow graph helped understand the logic behind cell values and that the interactions for navigating were intuitive. The participants mentioned several areas for improvement, including shortcuts to expand and collapse the whole flow graph, more visible expansion buttons and a more detailed function description when clicking a help button.

Several limitations constrain the study. First, the small sample size of five participants limits the generalizability and the study did not include sessions where participants used only traditional spreadsheet tools, which makes it impossible to know if the positive results come from the flow graph or from other factors. Also, the spreadsheet knowledge levels only ranged from 1 to

6, which gives no insight about the usability for expert users.

Future work could extend the system by adding support for a multi-level view, similar to the leveled dataflow diagrams by Hermans et al. [6]. This could help users navigate larger workbooks. Adding shortcuts for expanding and collapsing the flow graph as well as desynchronizing a cell could make it easier for the user to work with deeply nested formulas and keep everything organized. The help button could be improved so that the text is even more detailed and context-sensitive. The label inference mechanism could also be improved and used for more than labeling cell reference nodes. For example the labels could replace the pseudo-variables in binary and arithmetic nodes.

These extensions and improvements, combined with a comparative study, could provide stronger evidence for the effectiveness of the approach.

Bibliography

[1] HyperFormula (v3.1.1).

[2] ECMA-376-1:2016 Office Open XML File Formats - Fundamentals and Markup Language Reference, 2016.

[3] PAUL CHANDLER and JOHN SWELLER. THE SPLIT-ATTENTION EFFECT AS A FACTOR IN THE DESIGN OF INSTRUCTION. *British Journal of Educational Psychology*, 62(2):233–246, 1992. _eprint: https://bpspsychub.onlinelibrary.wiley.com/doi/pdf/10.1111/j.2044-8279.1992.tb01017.x.

[4] Hjalmar Gislason. Excel vs. Google Sheets usage — nature and numbers, January 2021.

[5] T.R.G. Green and M. Petre. Usability Analysis of Visual Programming Environments: A 'Cognitive Dimensions' Framework. *Journal of Visual Languages & Computing*, 7(2):131–174, June 1996.

[6] Felienne Hermans, Martin Pinzger, and Arie van Deursen. Supporting professional spreadsheet users by generating leveled dataflow diagrams. In *Proceedings of the 33rd International Conference on Software Engineering*, ICSE '11, pages 451–460, New York, NY, USA, 2011. Association for Computing Machinery.

[7] Felienne Hermans, Martin Pinzger, and Arie van Deursen. Detecting and visualizing inter-worksheet smells in spreadsheets. In *2012 34th International Conference on Software Engineering (ICSE)*, pages 441–451, June 2012.

[8] Karin Hodnigg, Christian Macho, Martin Pinzger, and Dietmar Jannach. Comprehending Spreadsheets: Which Strategies do Users Apply? In

2021 IEEE/ACM 29th International Conference on Program Comprehension (ICPC), pages 386–390, May 2021.

[9] T. Igarashi, J.D. Mackinlay, Bay-Wei Chang, and P.T. Zellweger. Fluid visualization of spreadsheet structures. In *Proceedings. 1998 IEEE Symposium on Visual Languages (Cat. No.98TB100254)*, pages 118–125, September 1998. ISSN: 1049-2615.

[10] Advait Sarkar, Andrew D. Gordon, Simon Peyton Jones, and Neil Toronto. Calculation View: multiple-representation editing in spreadsheets. In *2018 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, pages 85–93, October 2018.

[11] Ben Shneiderman. The eyes have it: A task by data type taxonomy for information visualizations. In *The craft of information visualization*, pages 364–371. Elsevier, 2003.

[12] Sruti Srinivasa Ragavan, Advait Sarkar, and Andrew D Gordon. Spreadsheet Comprehension: Guesswork, Giving Up and Going Back to the Author. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, pages 1–21, Yokohama Japan, May 2021. ACM.

[13] Melissa Rodriguez Zynda. The first killer app: a history of spreadsheets. *interactions*, 20(5):68–72, September 2013.

Dummy Appendix

You can defer lengthy calculations that would otherwise only interrupt the flow of your thesis to an appendix.

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. **In consultation with the supervisor**, one of the following two options must be selected:

☐ I hereby declare that I authored the work in question independently, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies[1].

☐ I hereby declare that I authored the work in question independently. In doing so I only used the authorised aids, which included suggestions from the supervisor regarding language and content and generative artificial intelligence technologies. The use of the latter and the respective source declarations proceeded in consultation with the supervisor.

Title of paper or thesis:

Authored by:
If the work was compiled in a group, the names of all authors are required.

Last name(s):	**First name(s):**

With my signature I confirm the following:

- I have adhered to the rules set out in the Citation Guidelines.
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

Place, date	**Signature(s)**

If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.

[1] For further information please consult the ETH Zurich websites, e.g. https://ethz.ch/en/the-eth-zurich/education/ai-in-education.html and https://library.ethz.ch/en/researching-and-publishing/scientific-writing-at-eth-zurich.html (subject to change).